Advancements in Artificial Intelligence for Network Intrusion Detection: A Comprehensive Analysis of Algorithmic Architectures, Empirical Benchmarks, and Operational Frameworks

Introduction to the Cybersecurity Landscape and the Critical Role of Intrusion Detection Systems

The modern digital era is characterized by an unprecedented expansion in data heterogeneity and volume, a phenomenon driven by the rapid maturation of the Internet of Things (IoT), 5G telecommunications, and multi-cloud architectures.[1] As network infrastructures become increasingly decentralized, the surface area for potential exploitation has grown exponentially, necessitating a transition from static perimeter defenses to dynamic, intelligent monitoring systems. Cybersecurity, at its core, is the practice of ensuring the confidentiality, integrity, and availability (CIA) of these digital assets.[3] Within this framework, Intrusion Detection Systems (IDS) serve as the vital sensory organ of a network, designed to analyze traffic patterns and system behaviors to identify deviations that signify malicious intent.[1]

The importance of IDS in modern enterprise environments cannot be overstated. With the global IoT security market projected to reach approximately $20.77 billion by 2025, the demand for scalable and adaptive security solutions is at an all-time high.[2] Traditional defense mechanisms, which often rely on manual oversight and deterministic logic, are increasingly paralyzed by the "Big Data" problem—the sheer scale of network logs and traffic flows makes human-only triage operationally untenable.[5] Furthermore, the rise of Advanced Persistent Threats (APTs) and sophisticated polymorphic malware requires a level of pattern recognition that exceeds the capabilities of standard rule-based filters.[4]

One of the most persistent hurdles in the field is the inherent imbalance found in network traffic data. In a typical enterprise environment, benign activities constitute more than 99% of all events, making the detection of rare malicious "needles" in the "haystack" of normal traffic a formidable statistical challenge.[7] This imbalance, coupled with the high-speed requirements of 5G and industrial control networks, creates a complex environment where an IDS must maintain near-perfect accuracy while operating with minimal latency.[2] AI-based Intrusion Detection Systems have emerged as a pivotal solution to these challenges, leveraging machine learning (ML) and deep learning (DL) algorithms to automatically learn, adapt, and respond to

the evolving threat landscape in real time.[1]

Problem Statement: The Technical Constraints of Traditional Detection and the Need for Intelligence

The primary objective of intrusion detection is the accurate classification of network events under conditions of extreme data volatility. However, the foundational methods of the previous decades are facing a crisis of utility. Traditional systems are largely dominated by Signature-based IDS (SIDS), which operate by matching observed traffic against a predefined database of known attack patterns.[11] While SIDS are highly efficient at identifying recognized threats with a low rate of false positives, they are inherently reactive. A SIDS is fundamentally blind to any activity for which it does not yet have a signature, a limitation that attackers exploit through zero-day vulnerabilities.[13]

The severity of the zero-day problem is evidenced by research showing that a typical zero-day attack remains undetected for an average of 312 days.[14] During this window, traditional security mechanisms are effectively bypassed, often leading to data breaches with an average cost of $1.2 million per incident.[14] Beyond the failure to detect unknown threats, traditional systems struggle with several operational bottlenecks:

The False Positive and False Negative Dilemma

Rule-based systems often generate a staggering volume of alerts, many of which are false positives (FPs) triggered by benign but unusual behavior, such as a large software update or a legitimate administrative task.[6] This leads to "alert fatigue," where security analysts become desensitized to warnings, potentially overlooking a genuine breach (a False Negative or FN) buried within the noise.[15] AI-driven models aim to mitigate this by establishing a more nuanced behavioral baseline, allowing for higher fidelity in detection.[6]

Scalability and Real-Time Latency Requirements

Modern networks process millions of packets per second. SIDS require a computational check against thousands of rules for every packet, which can cause significant performance degradation.[12] As exfiltration speeds increase—with the fastest modern attacks capable of stealing data within 72 minutes—the delay introduced by traditional processing becomes a critical security risk.[5] AI models must therefore be optimized for speed, balancing the complexity of deep learning architectures with the need for millisecond-level inference.[9]

Table 1: Comparison of SIDS Limitations and AI Solutions

Problem Category	Signature-Based (SIDS) Constraint	AI-Based IDS (Modern) Solution

Unknown Threats	Ineffective against zero-day exploits.	Anomaly detection identifies novel deviations.
Evasion Tactics	Bypassed by polymorphic/obfuscated code.	Analyzes behavioral intent rather than syntax.
Alert Management	High False Positive rate/Alert fatigue.	Behavioral baselining reduces noise.
Response Speed	Reactive (after-the-fact signature update).	Proactive and real-time detection.
Data Volume	Bottlenecks in high-speed networks.	Scalable through flow-based feature analysis.

5

Taxonomy of Intrusion Detection Systems: Architectures and Methodologies

Intrusion detection systems are categorized based on their detection logic and their physical or logical deployment within the network stack. Understanding these distinctions is crucial for designing a layered defense strategy.

Signature-Based IDS (SIDS)

SIDS function as a "digital handbook" of known malicious actors. They utilize string matching and packet header analysis to identify specific sequences of bytes or bit patterns associated with malware variants, SQL injection attempts, or unauthorized access requests.[11] Tools like Snort and Suricata are industry standards in this category. Their primary advantage is speed and reliability for known threats, but their static nature makes them useless against evolving payloads.[12]

Anomaly-Based IDS (AIDS)

AIDS represent the behavioral wing of intrusion detection. By using machine learning to establish a baseline of "normal" system behavior—including device communication patterns, data volume, and protocol usage—these systems can flag any notable departure as a potential threat.[11] This is conceptually similar to a security guard noticing a regular employee suddenly attempting to access the server room at 3:00 AM on a Sunday. While AIDS are superior for detecting zero-day exploits, they are prone to higher false-positive rates because "unusual" does not always mean "malicious".[1]

Hybrid Intrusion Detection Systems

The modern gold standard is a hybrid approach that integrates SIDS and AIDS. In these frameworks, a signature-based engine handles known attacks with high precision, while an AI-based anomaly engine monitors for subtle deviations.[3] This dual-stage detection ensures that security resources are used efficiently, reserving complex algorithmic processing for traffic that has already passed through initial deterministic filters.[20]

Network-Based (NIDS) vs. Host-Based (HIDS)

NIDS monitor traffic as it traverses the network, analyzing packets flowing between different nodes. They are effective at catching broad attacks like DDoS or network scanning.[22] HIDS, however, reside on individual servers or endpoints, monitoring internal system activities such as system calls, file integrity, and log files.[9] NIDS provide broad visibility, while HIDS provide the granular depth necessary to detect internal "living-off-the-land" attacks where a compromised legitimate process is used for malicious purposes.[5]

Table 2: Advantages and Limitations of IDS Types

Type	Advantage	Limitation
Signature-Based	High precision for known attacks; Low overhead.	Ineffective against zero-days; Requires frequent updates.
Anomaly-Based	Detects unknown and sophisticated threats.	Higher False Positive rate; Computationally expensive.
Hybrid	Combines speed of SIDS with	Complex to integrate and

	adaptability of AIDS.	manage.
Network-Based	Broad visibility; Does not affect host performance.	Blind to encrypted traffic or internal host actions.
Host-Based	Deep visibility into system-level activity.	Resource-intensive on the host; Limited network view.

5

Integrated Architecture of AI-Based Intrusion Detection Systems

The construction of an AI-based IDS follows a highly structured engineering pipeline, often compared to an automated assembly line where raw network data is methodically transformed into actionable security intelligence.[24]

Data Collection and Ingestion

The pipeline begins with the acquisition of raw data from diverse sources, such as routers, switches, and server logs.[25] This can occur via network taps or port mirroring (SPAN). Researchers distinguish between packet-level and flow-level data:

- **Packet-Based Capture**: Involves capturing the entire payload and header of every packet. This provides maximum visibility but is extremely resource-intensive and struggles with encrypted traffic.[22]
- **Flow-Based Capture**: Involves aggregating packets into "flows" or sessions (e.g., NetFlow data). A flow record typically includes the source/destination IP addresses, ports, protocol, and total bytes/packets.[21] Flow analysis is significantly more scalable and is the preferred method for backbone monitoring in high-speed environments.[22]

Preprocessing and Data Transformation

Raw network logs are rarely in a format suitable for machine learning. The preprocessing phase involves several essential steps:

1. **Cleaning**: Handling missing values and removing duplicate records. In early datasets like KDD Cup 99, redundant records caused models to overfit on common patterns, a flaw addressed in the subsequent NSL-KDD dataset.[3]

2. **Normalization/Scaling**: Since network features vary widely in magnitude (e.g., packet counts can be in the millions while duration is in fractions of a second), they must be scaled to a standard range, typically or a standard normal distribution (Z-score).[28]
3. **Encoding**: Categorical data, such as the protocol type (TCP, UDP, ICMP) or the service name, must be converted into numerical values through techniques like One-Hot Encoding.[3]

Feature Engineering and Selection

Feature engineering is arguably the most critical stage for model performance. It involves selecting the most relevant attributes and creating new ones that capture the temporal and behavioral characteristics of an attack.[31] For instance, a single packet might not look suspicious, but "50 packets sent to different ports on the same host within 2 seconds" is a clear indicator of a port scan.[33] Techniques like Recursive Feature Elimination (RFE) or Exhaustive Feature Selection (EFS) are used to identify the subset of features that maximizes detection accuracy while minimizing computational load.[29]

Model Training and Detection Layer

In this phase, the preprocessed data is used to train an AI model. The goal is for the model to learn the underlying patterns of both benign and malicious traffic. Once trained, the detection layer acts as the "brain" of the system, classifying incoming real-time traffic as either normal or a specific type of attack.[4]

Alerting and Mitigation

The final stage is the interface with the human analyst or an automated response system. High-fidelity alarms are generated and prioritized by threat severity.[6] In advanced systems, an autonomous response might be triggered—such as isolating a compromised host or updating a firewall rule—to contain the threat before "patient zero" results in a widespread breach.[6]

Exploration of Machine Learning and Deep Learning Models in IDS

AI-based intrusion detection leverages a spectrum of algorithmic families, from traditional "shallow" machine learning to complex "deep" neural networks.

Traditional Machine Learning (ML)

Traditional ML models are characterized by their reliance on manual feature engineering but offer the advantage of high speed and better interpretability for human analysts.[15]

- **Random Forest (RF)**: This ensemble method builds multiple decision trees and merges them to get a more accurate and stable prediction. It is particularly effective at capturing non-linear relationships in network data and is highly robust against the noise inherent in

real-world traffic.[11] Studies have shown RF achieving accuracies near 99% on balanced datasets.[35]

- **Support Vector Machines (SVM)**: SVMs find the optimal hyperplane that separates classes (e.g., Attack vs. Normal) with the largest possible margin. While effective for binary classification, they can be computationally intensive as the dataset size grows.[15] Optimized versions, like Least Square SVM (LS-SVM), are used in IDS to improve training efficiency.[29]
- **K-Nearest Neighbors (KNN)**: KNN classifies a traffic record based on the majority class of its "k" most similar neighbors in the feature space. While intuitive, it is sensitive to the scale of the data and can be slow in real-time if the reference dataset is large.[15]
- **Logistic Regression**: Often used as a baseline, it models the probability of a binary outcome. However, it often fails to capture the complex, multi-dimensional patterns of modern cyberattacks.[15]

Deep Learning (DL) Models

Deep learning eliminates much of the manual feature engineering by using multiple layers of artificial neurons to autonomously learn hierarchical representations of data.[15]

- **Convolutional Neural Networks (CNN)**: While famous for computer vision, CNNs are applied to IDS by treating network traffic as a 2D image or a spatial grid. They are excellent at detecting local patterns within a packet or flow.[15]
- **Recurrent Neural Networks (RNN) and LSTM**: Network traffic is inherently a sequence of events. RNNs, and specifically Long Short-Term Memory (LSTM) networks, are designed to process time-series data. They capture the temporal dependencies across packets, allowing them to detect stealthy, slow-moving attacks that occur over minutes or hours.[17]
- **Autoencoders (AE)**: These are unsupervised models used for anomaly detection. They learn to compress input data into a low-dimensional bottleneck and then reconstruct it. Because they are trained only on "normal" data, they struggle to reconstruct an "attack" input. This high "reconstruction error" serves as a precise signal for an intrusion.[19]

Table 3: Summary of Model Performance Characteristics

Model Family	Core Logic	IDS Application Strength	Primary Weakness
Random Forest	Decision Tree Ensemble	Robustness to noise; High accuracy.	Can be slow to predict with large ensembles.

SVM	Margin Maximization	Precision in high-dimensional data.	High training time for large datasets.
LSTM	Gated Recurrence	Captures long-term temporal patterns.	High computational cost; Hard to tune.
CNN	Spatial Filters	Identifies hierarchical local features.	Requires traffic data to be reshaped.
Autoencoders	Reconstruction Objective	Strong zero-day detection capability.	Prone to failure if "normal" data is noisy.

[15]

Performance Evaluation Metrics: Navigating the Imbalance Paradox

In the context of intrusion detection, a simple "accuracy" score can be dangerously misleading. If 99.9% of traffic is normal, a model that simply predicts "Normal" for everything will be 99.9% accurate while failing to detect every single attack.[7] Therefore, the cybersecurity community relies on more nuanced metrics derived from the confusion matrix.

Core Metrics and LaTeX Formulas

- **Accuracy**: The percentage of all predictions that were correct.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

[7]

- **Precision (Positive Predictive Value)**: The proportion of flagged alerts that were actually malicious. High precision means fewer false alarms.

$$Precision = \frac{TP}{TP + FP}$$

[7]

- **Recall (Sensitivity)**: The proportion of actual attacks that the system correctly identified. High recall means fewer missed attacks. This is often the most critical metric in security.[7]

$$Recall = \frac{TP}{TP + FN}$$

[7]

- **F1-Score**: The harmonic mean of precision and recall. It is the best single-number metric for imbalanced data, as it penalizes models that favor one metric at the expense of the other.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

[7]

- **ROC-AUC**: The Area Under the Curve of the Receiver Operating Characteristic plot. It shows the model's ability to distinguish between classes across all possible thresholds. An AUC of 0.5 is no better than a coin flip; 1.0 is perfect.[41]

The Criticality of Recall and F1-Score

In credit card fraud or intrusion detection, the cost of a False Negative (a missed attack) is far higher than the cost of a False Positive (a false alarm).[7] Therefore, researchers often prioritize **Recall** during model tuning. However, since too many false alarms can paralyze a security team, the **F1-Score** is used as a balancing mechanism to ensure the system remains operationally viable.[8]

Analysis of the State of the Art: Empirical Performance of Existing Systems

Recent research shows a clear trend toward hybrid and deep learning architectures that achieve state-of-the-art (SOTA) results across various benchmark datasets.

Model Comparisons and Trade-offs

A significant finding in the literature is that while Deep Learning (DL) models like LSTMs and CNNs often provide the highest raw accuracy and recall, traditional Machine Learning (ML) models like Random Forest remain competitive and are often more practical for real-time deployment due to their speed.[15]

- **Hybrid CapsNet + BiLSTM**: Combining Capsule Networks (for spatial feature preservation) with Bidirectional LSTM (for temporal dependencies) has achieved approximately 99% accuracy on CIC-IDS2017 and 97% on UNSW-NB15.[3]

- **Ensemble ML (RF-RFE)**: A model using Random Forest with Recursive Feature Elimination achieved 99.9% accuracy on CSE-CIC-IDS2018 with a fraction of the computational power required for deep neural networks.[35]
- **LS-SVM**: Optimized Support Vector Machines have demonstrated 99.3% accuracy on NSL-KDD, showing that with exhaustive feature selection, even "simpler" models can reach SOTA levels.[29]

Table 4: State-of-the-Art Performance Benchmark Summary

Model Architecture	Dataset	Accuracy	F1-Score	Inference Time
Hybrid CapsNet+BiLSTM	CIC-IDS2017	99.0%	0.99	Moderate
RF-RFE	CSE-CIC-IDS2018	99.9%	0.99	Very Fast
LS-SVM	NSL-KDD	99.3%	0.99	Fast
LSTM-Autoencoder	Edge-IIoTset	99.9%	0.99	Slow
XGBoost (Hybrid)	TII-SRC-23	99.95%	0.99	Fast

[2]

The fundamental trade-off in the SOTA is **Accuracy vs. Complexity**. While LSTMs are better at detecting rare, sequential attack patterns, their inference latency can be up to 75 times higher than optimized "lightweight" models.[9]

Evolution and Evaluation of IDS Datasets

The development of AI-based IDS is inextricably linked to the availability of high-quality, labeled datasets. These datasets act as the "textbooks" from which the algorithms learn.

KDD Cup 99 and NSL-KDD

KDD Cup 99 was the first major benchmark, containing millions of records from a simulated military network. However, it is now widely criticized for being outdated and containing significant redundancies that cause models to "cheat" by memorizing frequent normal traffic.[27] **NSL-KDD** was released in 2009 as a refined version, removing redundant records and rebalancing the test sets.[20] While still useful for pedagogy, it fails to represent modern web-based attacks or 21st-century protocols.[27]

UNSW-NB15

Created at the University of New South Wales, this dataset reflects more modern network traffic. It includes nine types of attacks, such as Fuzzers, Backdoors, and Worms. It is known for its high level of complexity and class overlap, which makes it a more realistic—and difficult—test for modern IDS.[20]

CICIDS2017 / 2018

The Canadian Institute for Cybersecurity released these datasets to provide a modern, comprehensive baseline. They include a week's worth of traffic with diverse attack types (DDoS, Botnets, Web Attacks, Brute Force) captured in a realistic network environment.[33] These are currently considered the most robust benchmarks for evaluating AI-based NIDS.[33]

Table 5: Characteristics and Limitations of Key IDS Datasets

Dataset	Created	Traffic Realism	Primary Features	Major Flaw
KDD Cup 99	1999	Simulated	41 attributes	Massive redundancy; Outdated.
NSL-KDD	2009	Refined KDD	41 attributes	Still based on 1998 traffic.

UNSW-NB15	2015	Real/Synthetic	47 attributes	High class overlap/complexity.
CICIDS2017	2017	Real-world	80 attributes	Large data volume (memory intensive).
CSE-CIC-2018	2018	Cloud-scale	80 attributes	Noisy; requires heavy cleaning.

20

Persistent Challenges and Constraints in AI-Based Detection

Despite the high accuracy reported in academic papers, operationalizing AI for intrusion detection faces significant hurdles.

Data Imbalance and Overfitting

The "imbalance" problem remains the single greatest challenge in data level training. If a model is not correctly balanced using techniques like SMOTE or ADASYN, it will inevitably overfit on the majority class, leading to a system that misses the very attacks it was designed to catch.[8]

Generalization to Evolving Attack Vectors

Models trained on static datasets often struggle to generalize to "out-of-distribution" data. In a real-world scenario, attackers are constantly changing their tactics. An AI model that is not continuously retrained can experience "model drift," where its accuracy slowly declines as the network environment and attack patterns evolve.[13]

Computational and Real-Time Constraints

Deploying deep learning models on resource-constrained devices, such as the ESP32 microcontrollers used in many IoT and industrial environments, is difficult. These models require substantial memory and processing power, necessitating the use of "edge-based" detection architectures where inference is decentralized.[9]

Data Poisoning and Adversarial ML

Attackers are now utilizing AI to break AI. By deliberately "poisoning" training data or creating adversarial examples—packets that look normal but contain malicious payloads designed to exploit the model's decision boundaries—threat actors can effectively blind an AI-based IDS.[10]

Proposed Solution: A Scalable, Multi-Stage IDS for Modern Networks

For organizations seeking to implement a state-of-the-art IDS (or for participants in a high-stakes hackathon), a modular, multi-stage architecture is recommended to balance accuracy and speed.

"Sentinel-AI" Architecture Blueprint

The proposed solution utilizes a hybrid approach, combining the speed of traditional ML with the deep analytical power of Autoencoders.

1. **Data Acquisition Layer**: Use **Scapy** for real-time packet capture and **CICFlowMeter** to extract 80+ behavioral features into NetFlow-style records.[23]
2. **Preprocessing Pipeline**: Implement an automated pipeline using **Scikit-learn**. This includes:
    - **Z-score Normalization** to handle high-variance features.
    - **ADASYN (Adaptive Synthetic Sampling)** to address class imbalance without over-generalizing like standard SMOTE.[44]
3. **Two-Stage Detection Engine**:
    - **Stage 1 (Binary Screening)**: An optimized **Random Forest** classifier acts as a high-speed filter. If the traffic is classified as "Normal" with high confidence (>95%), it is allowed through.[23]
    - **Stage 2 (Deep Anomaly Investigation)**: Any traffic flagged as suspicious or "low-confidence" is passed to a **Denoising Autoencoder**. Instead of a single Mean Squared Error (MSE) value, this model uses a **Reconstruction Error Vector**. This allows the system to identify exactly *which* feature is anomalous (e.g., an unusual packet-arrival interval), providing better diagnostic clarity.[39]
4. **Explainability and Alerting**: Integrate **SHAP** (Shapley Additive Explanations) to provide the SOC team with a clear visual representation of the features that contributed to an alert, reducing investigation time.[2]
5. **Deployment**: Use **Streamlit** for a real-time monitoring dashboard and **FastAPI** for a lightweight backend that can be containerized using Docker and deployed on Azure or AWS.[47]

Technological Recommendations

- **Core Language**: Python 3.9+.

- **Processing**: Pandas (batch), Apache Kafka (streaming).
- **Modeling**: PyTorch (for the Autoencoder), XGBoost (for high-speed classification).
- **Optimization**: Use **Grey Wolf Optimization (GWO)** for hyperparameter tuning to ensure the model achieves maximum F1-score with minimum training loss.[39]

Conclusion: The Strategic Imperative of AI in Cybersecurity

As cyberattacks move toward machine speed and exfiltration windows shrink to mere minutes, the transition to AI-based Intrusion Detection Systems is no longer optional—it is a strategic necessity. The traditional reliance on static signatures is fundamentally incompatible with a landscape defined by zero-day exploits and polymorphic threats.[11]

This report has detailed the progression from early SIDS architectures to modern hybrid and deep learning frameworks. While the performance of systems utilizing CapsNet, LSTMs, and Autoencoders has reached unprecedented levels—often exceeding 99% accuracy—the challenge remains one of operational integration. Success in the next decade of cybersecurity will depend on the ability to deploy "Lightweight" AI at the network edge, the mastery of data imbalance through adaptive resampling, and the development of explainable models that empower human analysts rather than overwhelming them with noise.[3] AI is not merely a tool for detection; it is the cornerstone of a proactive, resilient, and autonomous digital defense ecosystem.

Works cited

1. Artificial Intelligence based Intrusion Detection System – A Detailed Survey - ResearchGate, accessed April 7, 2026, https://www.researchgate.net/publication/382302921_Artificial_Intelligence_based_Intrusion_Detection_System_-_A_Detailed_Survey
2. AI-Enabled IoT Intrusion Detection: Unified Conceptual Framework and Research Roadmap, accessed April 7, 2026, https://www.mdpi.com/2504-4990/7/4/115
3. (PDF) Improving Intrusion Detection with Hybrid Deep Learning Models: A Study on CIC-IDS2017, UNSW-NB15, and KDD CUP 99 - ResearchGate, accessed April 7, 2026, https://www.researchgate.net/publication/389144460_Improving_Intrusion_Detection_with_Hybrid_Deep_Learning_Models_A_Study_on_CIC-IDS2017_UNSW-NB15_and_KDD_CUP_99
4. Deep Learning-based Intrusion Detection Systems: A Survey - arXiv, accessed April 7, 2026, https://arxiv.org/html/2504.07839v3
5. AI threat detection: 7 methods across 6 security domains - Vectra AI, accessed April 7, 2026, https://www.vectra.ai/topics/ai-threat-detection
6. AI Network Security Protection - Darktrace, accessed April 7, 2026, https://www.darktrace.com/products/network
7. Classification: Accuracy, recall, precision, and related metrics | Machine Learning,

accessed April 7, 2026, https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall
8. Addressing Imbalanced Data in Network Intrusion Detection: A Review and Survey, accessed April 7, 2026, https://www.researchgate.net/publication/378721081_Addressing_Imbalanced_Data_in_Network_Intrusion_Detection_A_Review_and_Survey
9. LIGHT-HIDS: A Lightweight and Effective Machine Learning-Based Framework for Robust Host Intrusion Detection - arXiv, accessed April 7, 2026, https://arxiv.org/html/2509.13464v1
10. The Impact of Machine Learning on Intrusion Detection Accuracy | OrhanErgun.net Blog, accessed April 7, 2026, https://orhanergun.net/the-impact-of-machine-learning-on-intrusion-detection-accuracy
11. A survey paper on AI Based Intrusion Detection System - ResearchGate, accessed April 7, 2026, https://www.researchgate.net/publication/388799447_A_survey_paper_on_AI_Based_Intrusion_Detection_System
12. Overview on Intrusion Detection Systems for Computers Networking Security - MDPI, accessed April 7, 2026, https://www.mdpi.com/2073-431X/14/3/87
13. A review of Machine Learning-based zero-day attack detection: Challenges and future directions - ResearchGate, accessed April 7, 2026, https://www.researchgate.net/publication/365734823_A_review_of_machine_learning-based_zero-day_attack_detection_Challenges_and_future_directions
14. A Survey of Machine Learning-Based Zero-Day Attack Detection: Challenges and Future Directions - PMC, accessed April 7, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC9890381/
15. Deep Learning vs. Machine Learning for Intrusion Detection in ..., accessed April 7, 2026, https://www.mdpi.com/2076-3417/15/4/1903
16. Darktrace Competitor Comparison - Vectra AI, accessed April 7, 2026, https://www.vectra.ai/products/competitive/darktrace
17. Evaluating the Performance of Machine Learning and Deep Learning Algorithms in Network Intrusion Detection Systems (NIDS) - ResearchGate, accessed April 7, 2026, https://www.researchgate.net/publication/396394252_Evaluating_the_Performance_of_Machine_Learning_and_Deep_Learning_Algorithms_in_Network_Intrusion_Detection_Systems_NIDS
18. Zero-Day Attack AI-based IDS - ESP Journals, accessed April 7, 2026, https://www.espjournals.org/IJCSIT/2025/Volume1-Issue2/IJCSIT-V1I2P104.pdf
19. Autoencoders for Anomaly Detection: Intuition to Code | Let's Data Science, accessed April 7, 2026, https://letsdatascience.com/blog/the-art-of-failing-gracefully-finding-anomalies-with-autoencoders
20. View of Improving Intrusion Detection with Hybrid Deep Learning ..., accessed April 7, 2026, https://jisem-journal.com/index.php/journal/article/view/1665/653

21. Flow-Based and Packet-Based Intrusion Detection ... - SMU Scholar, accessed April 7, 2026, https://scholar.smu.edu/cgi/viewcontent.cgi?article=1162&context=datasciencereview
22. Packet Vs. Flow: A Look at Network Traffic Analysis Techniques - LiveAction, accessed April 7, 2026, https://www.liveaction.com/resources/solution-briefs/packet-vs-flow-a-look-at-network-traffic-analysis-techniques/
23. How to Build a Real-Time Intrusion Detection System with Python and Open-Source Libraries - freeCodeCamp, accessed April 7, 2026, https://www.freecodecamp.org/news/build-a-real-time-intrusion-detection-system-with-python/
24. Machine Learning Pipeline Architecture: A Practical Guide to Design and Scale | DSG.AI, accessed April 7, 2026, https://www.dsg.ai/blog/machine-learning-pipeline-architecture
25. What Is a Machine Learning Pipeline? - Snowflake, accessed April 7, 2026, https://www.snowflake.com/en/fundamentals/building-effective-machine-learning-pipelines/
26. Comparison of feature extraction tools for network traffic data - arXiv, accessed April 7, 2026, https://arxiv.org/pdf/2501.13004
27. The KDD Dataset: A Beginner's Guide to the Most Famous Benchmark in Intrusion Detection, accessed April 7, 2026, https://medium.com/@lazybrainz/the-kdd-dataset-a-beginners-guide-to-the-most-famous-benchmark-in-intrusion-detection-49765f83adf4
28. Optimized intrusion detection in IoT and fog computing using ensemble learning and advanced feature selection - PMC, accessed April 7, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC11293719/
29. Intrusion detection system based on machine learning using least ..., accessed April 7, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC11978955/
30. Packet-Level and Flow-Level Network Intrusion Detection Based on Reinforcement Learning and Adversarial Training - MDPI, accessed April 7, 2026, https://www.mdpi.com/1999-4893/15/12/453
31. A Beginner's Guide for Feature Engineering in Machine Learning - Tredence, accessed April 7, 2026, https://www.tredence.com/blog/feature-engineering-in-machine-learning
32. Machine Learning Pipelines: Data Preprocessing, Feature Engineering, and Model Training | by Dhairya Saini | Medium, accessed April 7, 2026, https://medium.com/@dhairyasaini1805/machine-learning-pipelines-data-preprocessing-feature-engineering-and-model-training-0c45691afbec
33. Network intrusion detection using a hybrid graph-based ... - PMC - NIH, accessed April 7, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12822977/
34. Full article: Intrusion detection using graph inspired insights, accessed April 7, 2026, https://www.tandfonline.com/doi/full/10.1080/1206212X.2025.2584106
35. Optimization of predictive performance of intrusion detection system using hybrid ensemble model for secure systems - PMC, accessed April 7, 2026,

https://pmc.ncbi.nlm.nih.gov/articles/PMC10496009/
36. Cisco Secure Network Analytics (formerly Stealthwatch) At-a-Glance, accessed April 7, 2026, https://www.cisco.com/c/en/us/products/collateral/security/stealthwatch/secure-network-analytics-aag.html
37. Mitigating class imbalance in churn prediction with ensemble methods and SMOTE - PMC, accessed April 7, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12064797/
38. Vanilla Autoencoders for Anomaly Detection | by Sudeep Chavare - Medium, accessed April 7, 2026, https://medium.com/@sudeepdc/vanilla-autoencoders-for-anomaly-detection-b00919a95e70
39. (PDF) Practical autoencoder based anomaly detection by using ..., accessed April 7, 2026, https://www.researchgate.net/publication/366852853_Practical_autoencoder_based_anomaly_detection_by_using_vector_reconstruction_error
40. Autoencoders for Anomaly Detection are Unreliable - arXiv, accessed April 7, 2026, https://arxiv.org/html/2501.13864v1
41. Evaluation Metrics in Machine Learning - GeeksforGeeks, accessed April 7, 2026, https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/
42. Performance Metrics: Confusion matrix, Precision, Recall, and F1 Score, accessed April 7, 2026, https://towardsdatascience.com/performance-metrics-confusion-matrix-precision-recall-and-f1-score-a8fe076a2262/
43. Metrics for Classification Model - AlmaBetter, accessed April 7, 2026, https://www.almabetter.com/bytes/tutorials/data-science/classification-metrics
44. Investigating Oversampling Techniques to Mitigate Class Imbalance in Network Intrusion Detection Datasets - Scilight Press, accessed April 7, 2026, https://media.sciltp.com/articles/2601002807/2601002807.pdf
45. Real-time temperature anomaly detection in vaccine refrigeration systems using deep learning on a resource-constrained microcontroller - Frontiers, accessed April 7, 2026, https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2024.1429602/full
46. Constrained Concealment Attacks against Reconstruction-based Anomaly Detectors in Industrial Control Systems - CISPA, accessed April 7, 2026, https://publications.cispa.saarland/3226/1/Real_time_Evasion_Attacks.pdf
47. streamlit Applications - Lablab.ai, accessed April 7, 2026, https://lablab.ai/apps/tech/streamlit
48. How to Build a Real-Time Intrusion Detection System with Python and Open-Source Libraries - Chaitanya Rahalkar, accessed April 7, 2026, https://rahalkar.dev/posts/2025-01-31-realtime-intrusion-detection-python/
49. A Comparative Review of SMOTE and ADASYN in Imbalanced Data Classification, accessed April 7, 2026,

https://www.semanticscholar.org/paper/A-Comparative-Review-of-SMOTE-and-ADASYN-in-Data-Brandt-Lanz%C3%A9n/814bbf5f41a501c0b415e26a48cf5e9a5875a934
50. Structured Denoising Autoencoder for Fault Detection and Analysis - Proceedings of Machine Learning Research, accessed April 7, 2026, http://proceedings.mlr.press/v39/tagawa14.pdf
51. Streamlit Meets Scenarios: Building Adaptive AI for the Enterprise - YouTube, accessed April 7, 2026, https://www.youtube.com/watch?v=qJFsN3qiXTQ